BÁO CÁO THỰC HÀNH LAB 01

EMBEDDED LINUX SYSTEMS | ARMv7 Cortex-A9 | QEMU vexpress-a9

Cấu hình Linux Kernel, U-Boot, BusyBox và khởi động hệ thống Linux nhúng

Thông tin	Nội dung
Sinh viên	Nguyễn Thanh Phong
MSSV	SE182514
Bài thực hành	LAB-01
Nền tảng	ARMv7-A Cortex-A9 / QEMU vexpress-a9
Ngày tổng hợp minh chứng	30/10/2026
Nhóm	Chưa được cung cấp

Phạm vi báo cáo: Tổng hợp từ ảnh chụp Terminal trong tệp bao_cao.zip, kết quả lệnh và boot log đã cung cấp trong cuộc trò chuyện. Không giả định các bước chưa có bằng chứng là đã hoàn tất.

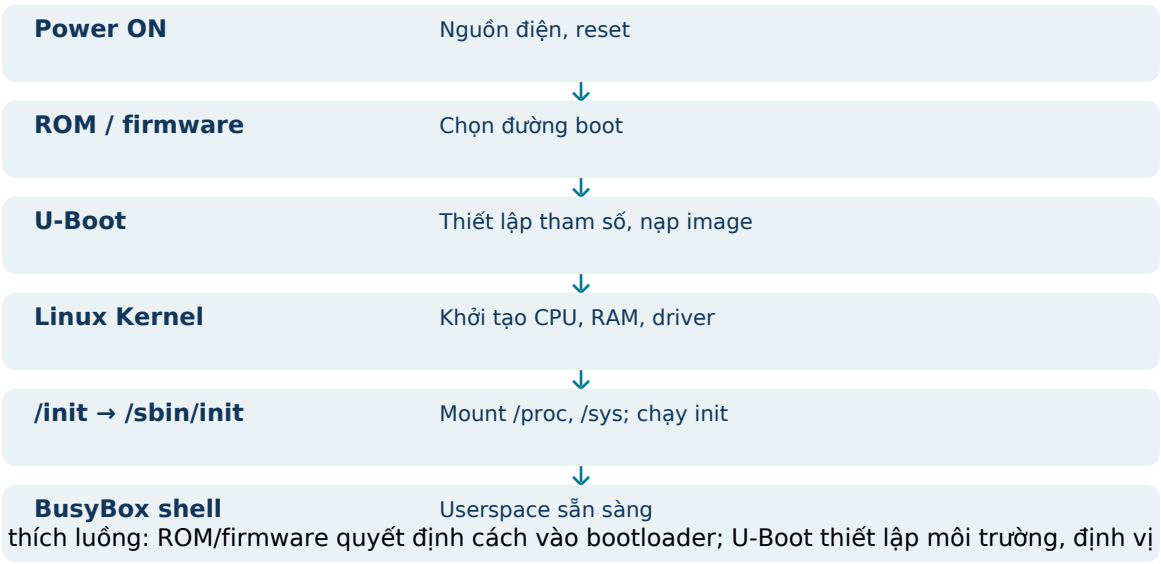
Kết quả chính: Kernel 5.15, U-Boot 2022.04 và BusyBox ARM 32-bit được xác minh; QEMU đã vào được BusyBox shell. Đã quan sát Kernel panic khi không nạp root filesystem. Lệnh saveenv chưa thành công.

1. Mục tiêu và môi trường thực hành

Thực hành cross-compile Linux Kernel cho ARMv7, kiểm tra bootloader U-Boot, xây dựng root filesystem tối giản bằng BusyBox static và chạy Linux trên QEMU Versatile Express Cortex-A9.

Thành phần	Thông tin kiểm tra
Kernel	Linux 5.15.0; zImage ~4.9 MB; DTB ~14 KB
U-Boot	2022.04; ELF 32-bit ARM; u-boot ~4.2 MB
BusyBox	ELF 32-bit ARM EABI5; statically linked; initramfs ~1.1 MB
Toolchain đang cài	arm-linux-gnueabihf-gcc 15.2.0
Lưu ý về compiler	Boot log kernel ghi arm-linux-gnueabi-gcc; khác toolchain kiểm tra hiện tại

2. Sơ đồ trình tự khởi động (Boot Sequence)



Giải thích luồng: ROM/firmware quyết định cách vào bootloader; U-Boot thiết lập môi trường, định vị kernel và Device Tree, có thể chuyển initramfs cho kernel. Kernel giải nén, khởi tạo bộ nhớ, thiết bị và rootfs, sau đó chạy /init (hoặc rdinit), chuyển sang /sbin/init và BusyBox shell.

Phân biệt hai phép thử: Trong thí nghiệm U-Boot, lệnh printenv/setenv được chạy tại dấu nhắc =>. Trong phép thử khởi động Linux thành công, QEMU được gọi trực tiếp bằng -kernel, -dtb và -initrd; dữ liệu hiện có chưa chứng minh chuỗi U-Boot → Linux được thực thi hoàn chỉnh trong cùng một phiên.

3. Kết quả thực hành

3.1. Phần A - Linux Kernel

File zImage và vexpress-v2p-ca9.dtb đã tồn tại. Cấu hình hiển thị CONFIG_BLK_DEV_INITRD=y, CONFIG_ARCH_VEXPRESS=y, CONFIG_CPU_V7=y và CONFIG_SMP=y. Lệnh file nhận diện Linux kernel ARM boot executable zImage; readelf cho Tag_CPU_arch: v7 và Tag_CPU_name: 7-A.

Điểm cần lưu ý: Ảnh chụp hiện tại xác nhận sự tồn tại và cấu hình của bản build, không bao gồm toàn bộ log cuối lệnh make. Boot log bản Kernel 5.15 ghi trình biên dịch arm-linux-gnueabi-gcc 15.2.0; vì thế không thể khẳng định bản kernel thực tế đã được build bằng gnueabihf chỉ từ lệnh kiểm tra phiên bản compiler trên máy.

3.2. Phần B - U-Boot

U-Boot 2022.04 khởi động trên QEMU với DRAM 512 MiB, Flash 64 MiB và đã vào được dấu nhắc =>. Lệnh printenv cho thấy board=vexpress, cpu=armv7, fdtfile=vexpress-v2p-ca9.dtb, kernel_addr_r=0x60100000 và fdt_addr_r=0x60000000.

Sau setenv, kết quả kiểm tra là bootargs=console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M và bootcmd=bootz 0x60100000 0x64000000 0x60000000. Chưa có bằng chứng các image đã được nạp tại ba địa chỉ này nên bootcmd mới được xác nhận ở mức giá trị biến, chưa xác nhận thực thi thành công.

Lệnh saveenv trả về: Error: start and/or end address not on sector boundary; Failed (1). Vì thế cấu hình trong phiên đó không được lưu bền vững vào Flash.

3.3. Phần C - BusyBox và initramfs

Binary rootfs/initramfs/bin/busybox là ELF 32-bit ARM EABI5, statically linked; CONFIG_STATIC=y. Script /init mount proc, sysfs rồi exec /sbin/init. QEMU đã khởi động tới / #; lệnh ps thấy tiến trình init PID 1 và shell /bin/sh. Kết quả cpuinfo trong ảnh cho thấy ARMv7 Processor và ARM-Versatile Express.

CONFIG_CROSS_COMPILER_PREFIX="" trong cấu hình BusyBox: đây chỉ là giá trị cấu hình lưu trong file; binary ARM đã có, nhưng cách gọi công cụ build ban đầu chưa được chứng minh bằng ảnh.

3.4. Phép thử thiếu root filesystem

Khi QEMU được chạy mà không có -initrd, log ghi: “VFS: Cannot open root device ram0 or unknown-block(0,0)” và “Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)”. Đây là kết quả thực tế, khác ví dụ unknown-block(1,0) trong đề. Kernel không thể mount rootfs để chạy init.

4. Trả lời 5 câu hỏi phân tích

Câu 1. gnueabi và gnueabihf

arm-linux-gnueabi và arm-linux-gnueabihf khác chủ yếu ở quy ước ABI đối với đối số/giá trị dấu phẩy động: biến thể hard-float sử dụng thanh ghi VFP khi truyền đối số float theo ABI tương ứng, còn soft-float ABI truyền theo thanh ghi nguyên/stack. Cortex-A9 có thể hỗ trợ VFP, nên dùng hard-float cho userspace khi hệ thống được cấu hình FPU phù hợp sẽ tận dụng ABI đó. Tuy nhiên, không phải cứ dùng gnueabi là kernel không chạy: kernel ARM được build bằng một toolchain gnueabi phù hợp vẫn có thể boot, vì quy tắc ABI của chương trình userspace khác việc kernel truy cập FPU. Điều quan trọng là tương thích ABI giữa binary userspace và thư viện/loader tương ứng. Dữ liệu thực tế: log kernel đang ghi compiler gnueabi, còn BusyBox ARM static vẫn chạy được.

Câu 2. Memory map và các địa chỉ boot

Bản đồ bộ nhớ của board được QEMU/thiết kế Versatile Express quy định: trong boot log có RAM 0x60000000–0x7fffffff, không bắt đầu tại 0x00000000 như platform Versatile PB. Các địa chỉ bootcmd phải nằm trong RAM hợp lệ và không ghi đè lẫn nhau: DTB 0x60000000, zImage 0x60100000, initramfs 0x64000000. Đặt sai địa chỉ, hoặc để image chồng lấn/vùng dành riêng, có thể gây lỗi nạp, lỗi giải nén, lỗi Device Tree, đứng máy hoặc kernel panic. Giá trị biến bootcmd tự nó chưa chứng minh dữ liệu được nạp đúng vào RAM.

Câu 3. MIDR 0x410fc090

MIDR là Main ID Register của CPU. Với 0x410fc090: implementer [31:24]=0x41 (Arm); variant [23:20]=0; architecture [19:16]=0xF (trường architecture dùng mã theo cơ chế định danh của ARM); part number [15:4]=0xC09, tương ứng Cortex-A9; revision [3:0]=0. Vì vậy log “ARMv7 Processor [410fc090] revision 0” phù hợp với Cortex-A9 revision 0.

Câu 4. Static linking của BusyBox

Static linking đóng gói mã thư viện cần thiết vào binary giúp rootfs tối giản không phụ thuộc loader động và thư viện runtime trong /lib. Nếu liên kết động bằng GNU libc cho ARM hard-float, thường cần dynamic loader ld-linux-armhf.so.3, libc.so.6 và tùy applet/cấu hình có thể cần libm.so.6 hoặc libresolv.so.2. Trên máy Ubuntu dùng cross-toolchain, các file hay nằm trong /usr/arm-linux-gnueabihf/lib hoặc thư mục sysroot tương ứng; phải kiểm tra thực tế bằng readelf -l và readelf -d trước khi copy. Ảnh cho thấy bản BusyBox đang dùng là statically linked nên không cần chép các thư viện này.

Câu 5. SMP khi dùng -smp 2

SMP cần khởi tạo CPU thứ cấp, thiết lập per-CPU data, timer, interrupt controller, scheduler và đồng bộ liên CPU. Với 2 CPU hoạt động, boot log thông thường có thông điệp secondary CPUs brought up và SMP: Total of 2 processors activated; sysfs online thường là 0-1. Trong bản log thử thiếu initramfs đã cung cấp có “smp: Brought up 1 node, 1 CPU” và “SMP: Total of 1 processors activated”, phù hợp phiên QEMU không chỉ định -smp 2. Ảnh shell ở phép thử trước cho thấy cpuinfo có hai mục processor, nhưng hiện chưa có hai boot log đủ điều kiện đối chiếu trực tiếp -smp 1 với -smp 2; hai dòng so sánh cụ thể này cần chụp bổ sung nếu giảng viên yêu cầu bằng chứng chênh lệch.

5. Nhận xét, khó khăn và giải pháp

Vấn đề quan sát	Phân tích và hướng xử lý
U-Boot tự boot thất bại	Distro boot thử MMC/TFTP không tìm thấy image phù hợp; vào được shell =>, có thể kiểm tra environment, cấu hình nguồn nạp thực tế và địa chỉ ảnh trước khi boot.
saveenv thất bại	Vùng Flash lưu environment chưa căn chỉnh theo ranh giới sector; chưa lưu bền vững. Giải pháp: kiểm tra cấu hình flash/env offset/size và layout QEMU trước khi cấu hình lưu.
Kernel panic khi thiếu initramfs	Không có rootfs để mount; khi thêm -initrd hợp lệ, hệ thống đã có thể chạy vào BusyBox shell.

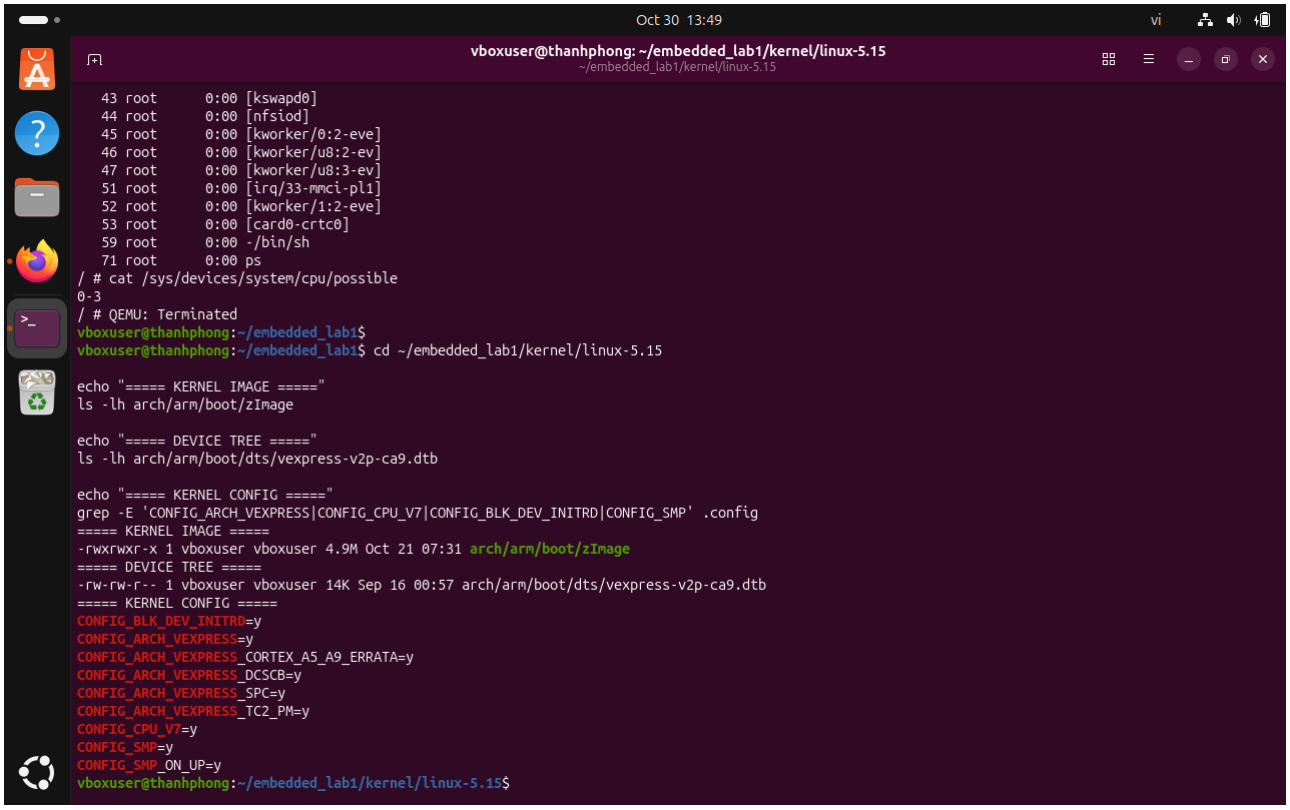
6. Kết luận và công việc còn lại

Các bằng chứng hiện có cho thấy xây dựng được kernel ARMv7, U-Boot ARMv7 và BusyBox static; QEMU boot trực tiếp vào BusyBox shell. Báo cáo không tuyên bố U-Boot đã load và boot kernel thành công hoặc saveenv đã lưu được dữ liệu. Cần bổ sung nếu muốn đầy đủ tuyệt đối: log make build, đối chứng -smp 1/-smp 2, xác nhận compiler kernel và thông tin nhóm sinh viên.

Hình minh chứng: Các ảnh màn hình trong phụ lục được giữ nguyên từ ZIP người dùng cung cấp, không chỉnh sửa kết quả thực hành.

PHỤ LỤC - ẢNH MINH CHỨNG THỰC HÀNH

A1. Kernel image, Device Tree và các tùy chọn cấu hình



Nguồn: Lab01_PhanA_01_Kernel_Config.png

A2. Xác nhận định dạng ARM zImage và Tag_CPU_arch



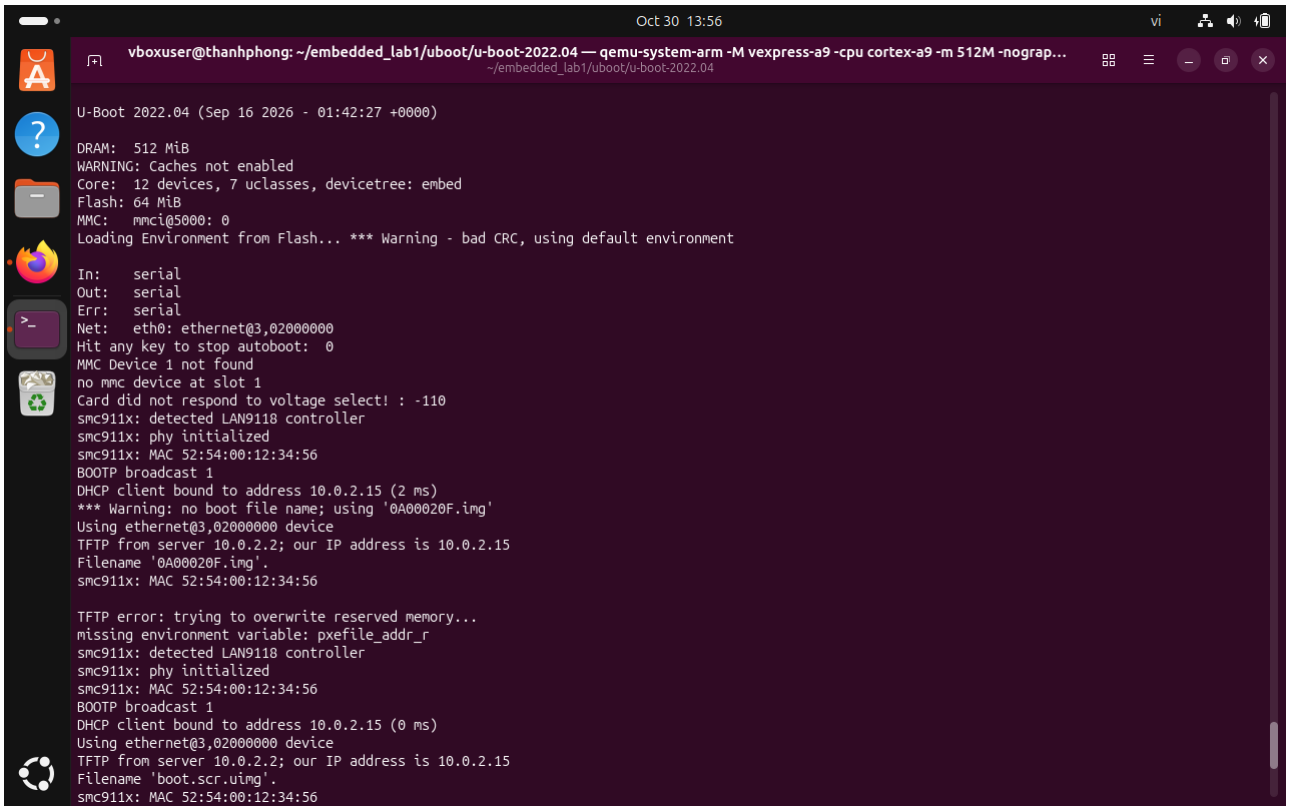
Nguồn: Lab01_PhanA_02_Kernel_Architecture.png

B1. Các file U-Boot và kiến trúc ARMv7



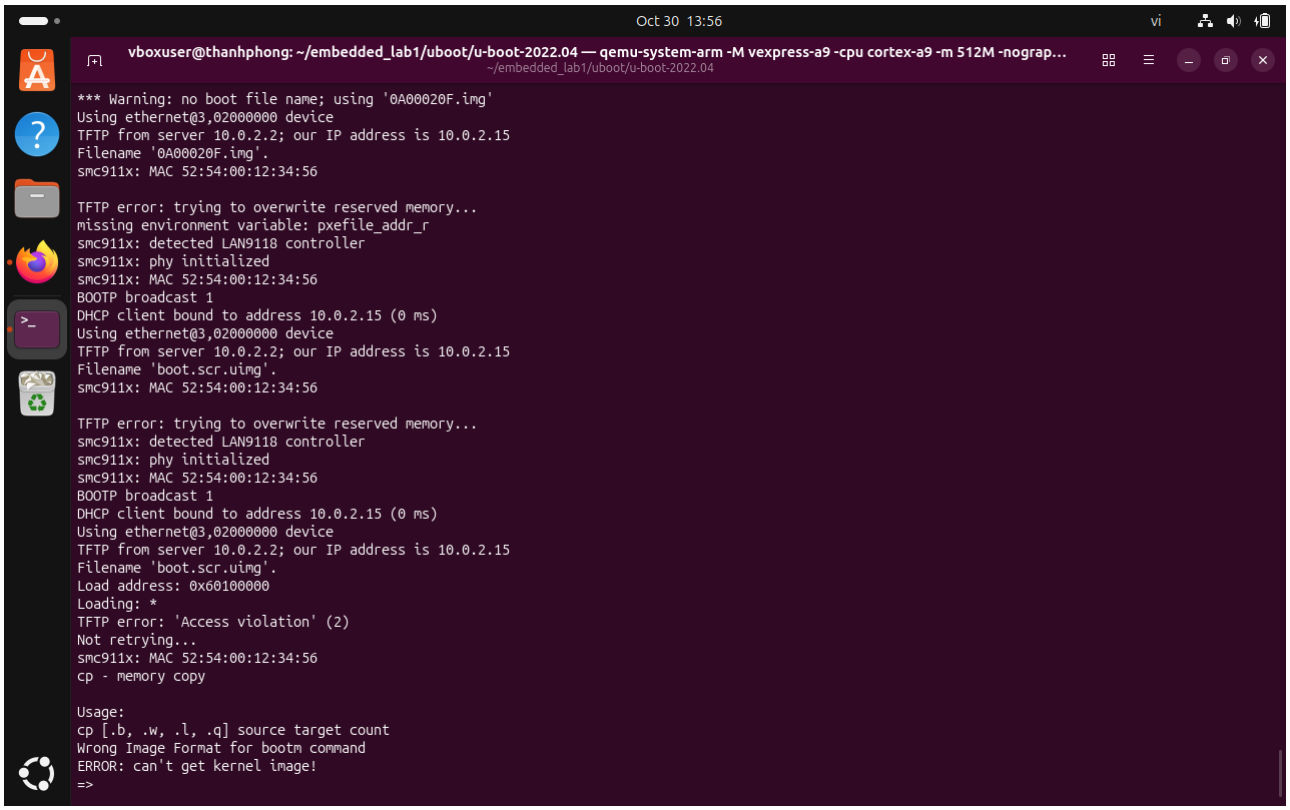
Nguồn: Lab01_PhanB_01_UBoot_Architecture.png

B2a. Khởi động U-Boot trên QEMU



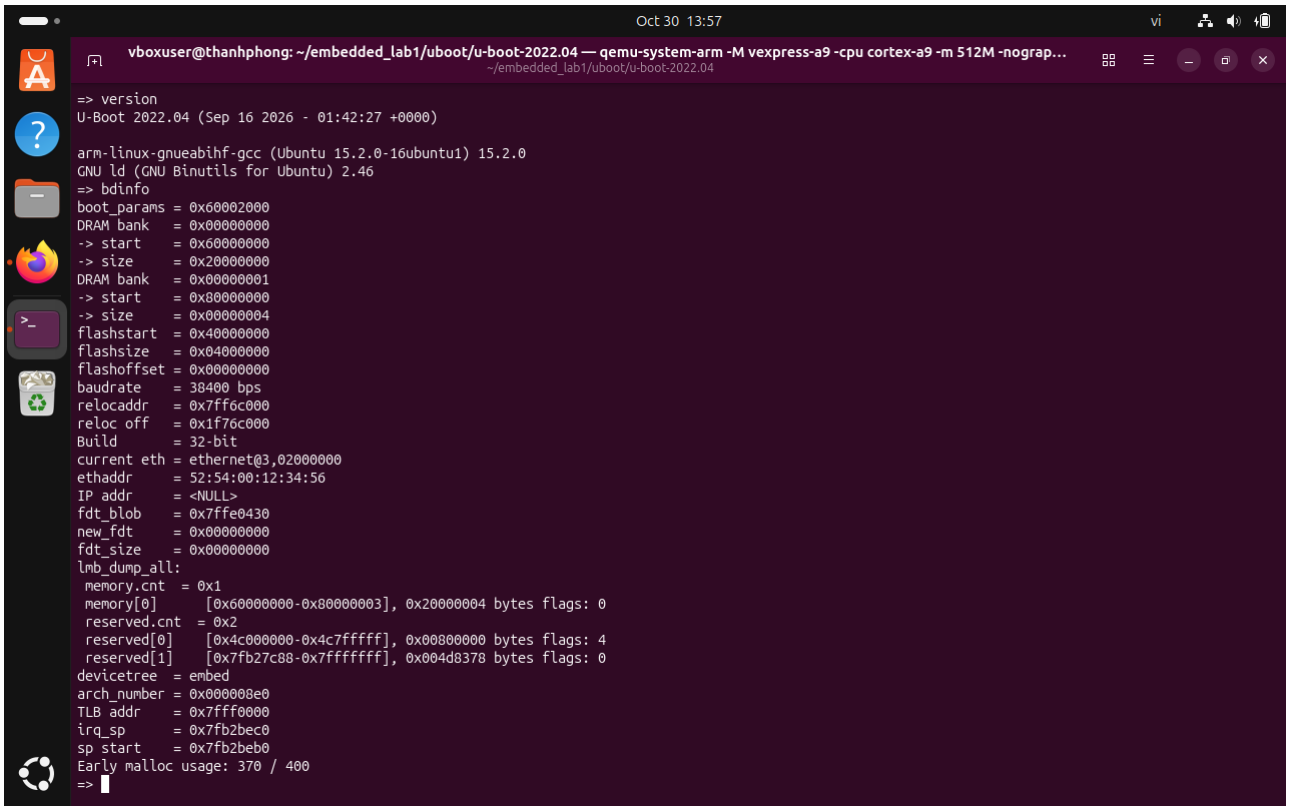
Nguồn: Lab01_PhanB_02_UBoot_Boot1.png

B2b. Nhật ký auto boot và U-Boot shell



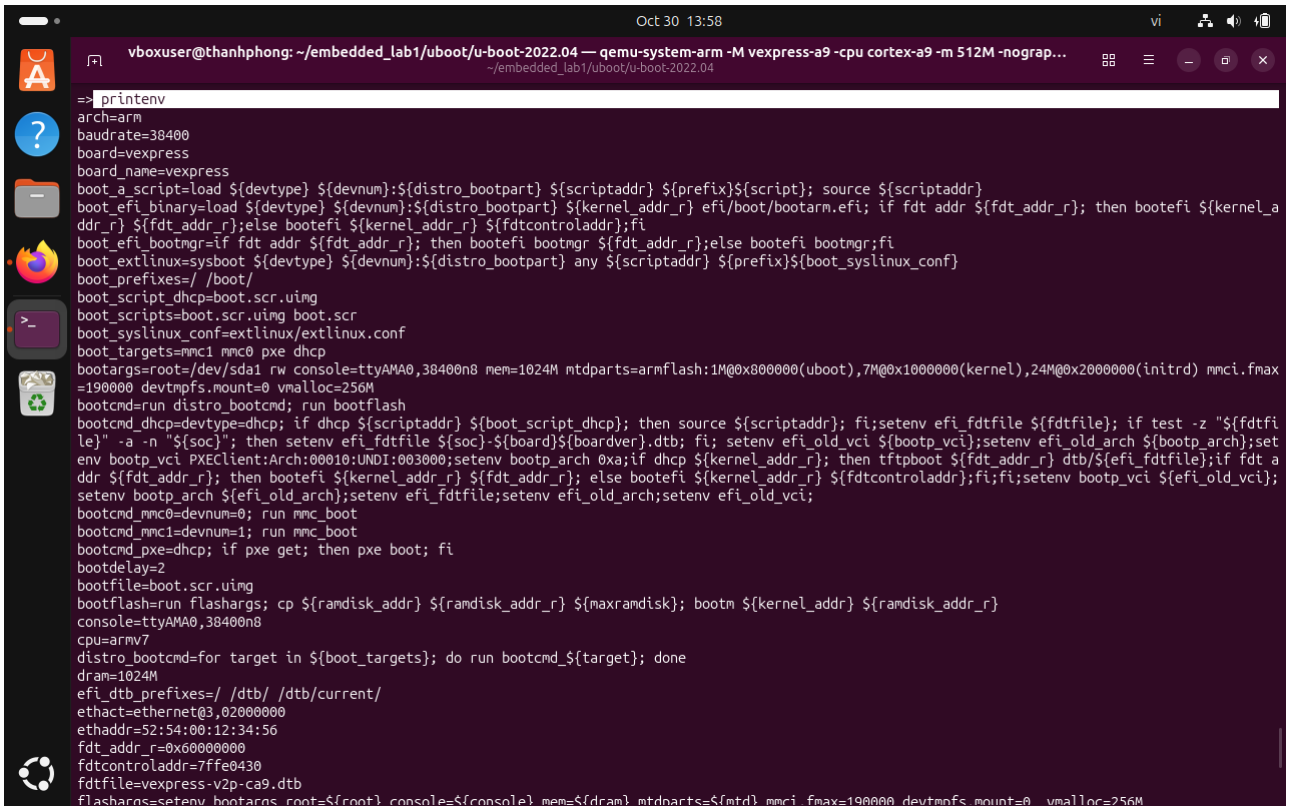
Nguồn: Lab01_PhanB_02_UBoot_Boot2.png

B3. Thông tin board - version/bdinfo



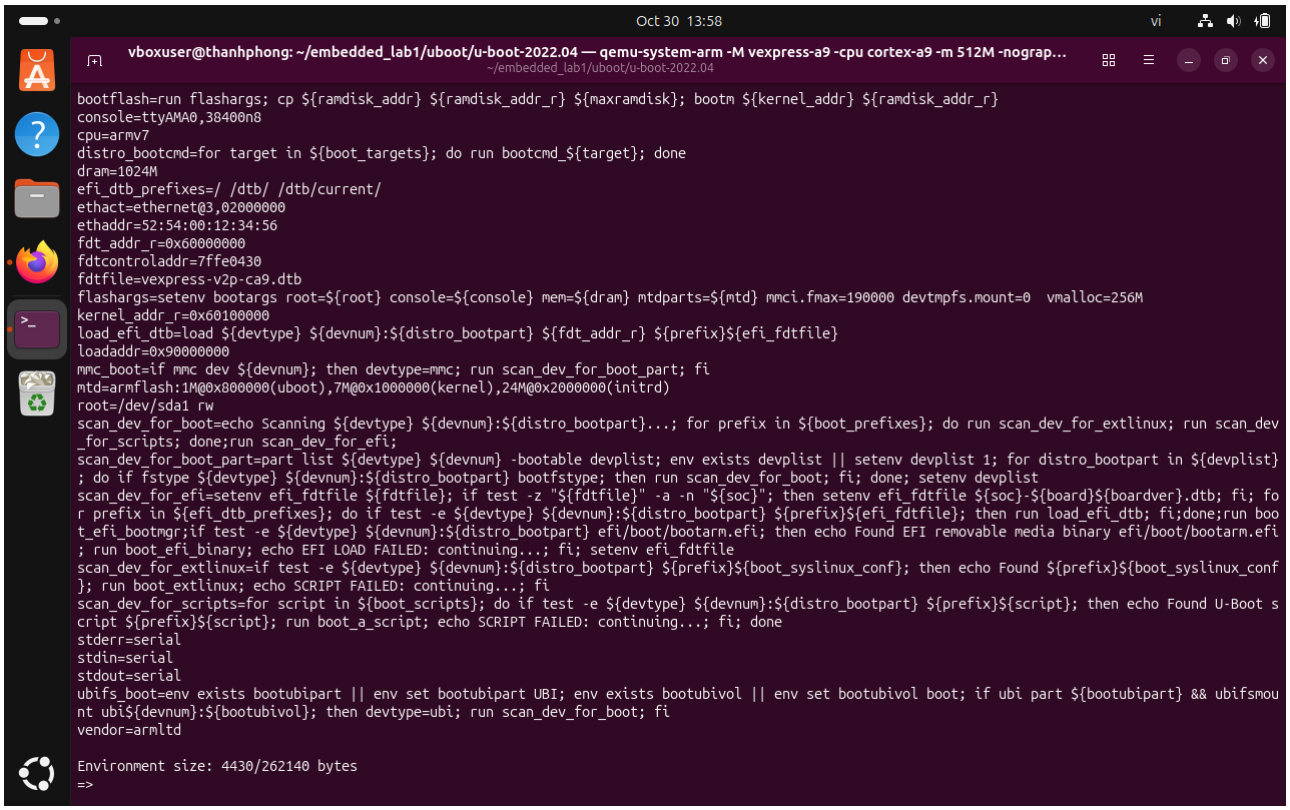
Nguồn: Lab01_PhanB_03_UBoot_Info.png

B4a. printenv (phần 1)



Nguồn: Lab01_PhanB_04_UBoot_Environment1.png

B4b. printenv (phần 2)



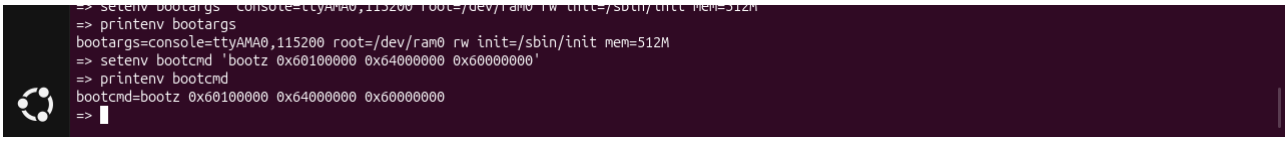
Nguồn: Lab01_PhanB_04_UBoot_Environment2.png

B5. Thiết lập bootargs



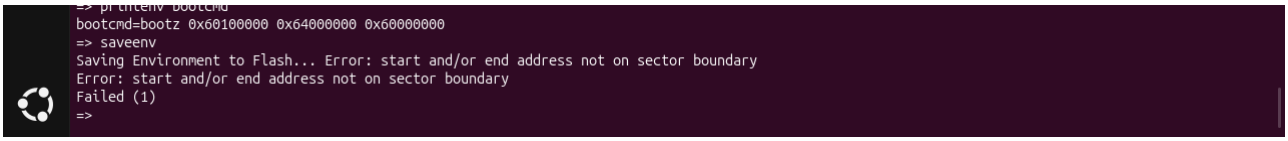
Nguồn: Lab01_PhanB_05_Bootargs.png

B6. Thiết lập bootcmd



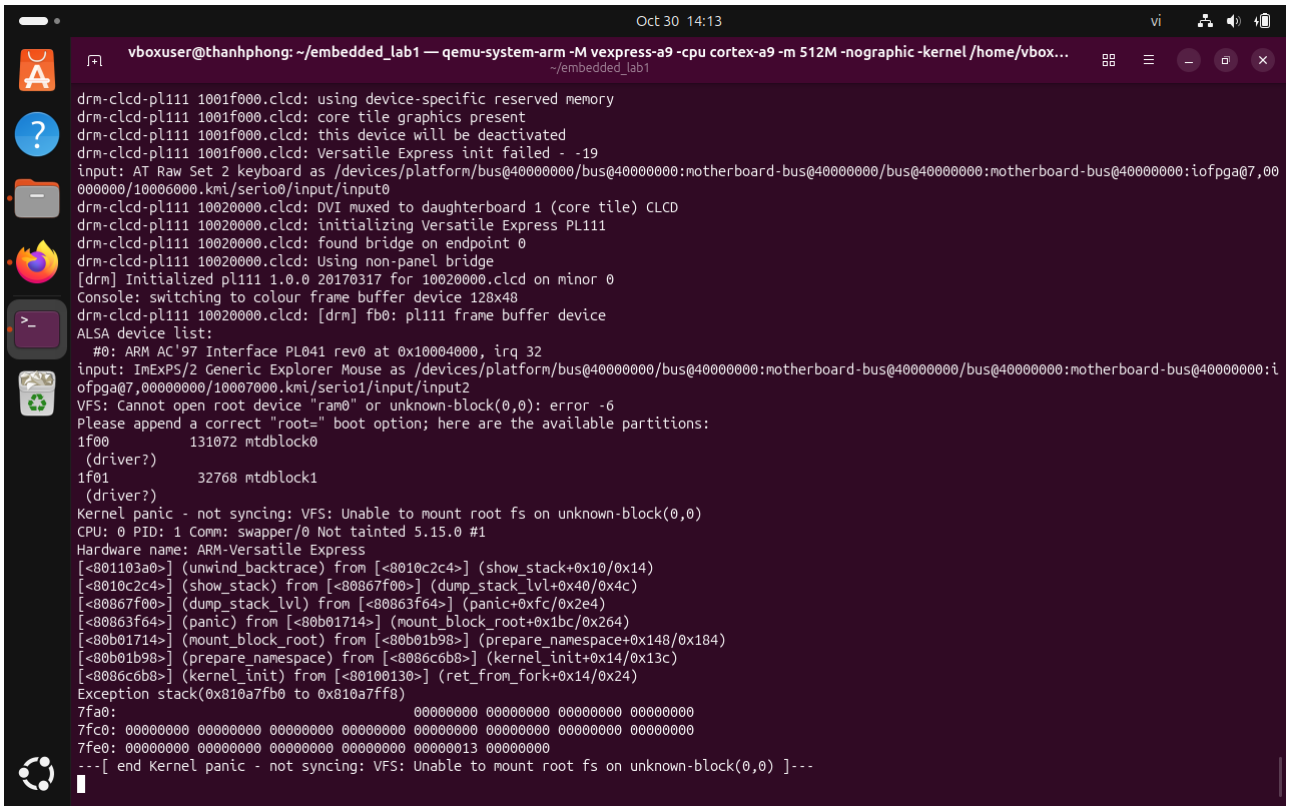
Nguồn: Lab01_PhanB_06_Bootcmd1.png

B7. saveenv không thành công



Nguồn: Lab01_PhanB_06_Saveenv_Error.png

B8. Kernel panic khi không có initramfs



Nguồn: Lab01_PhanB_07_Kernel_Panic.png

C1. Kernel và thông tin CPU ARMv7

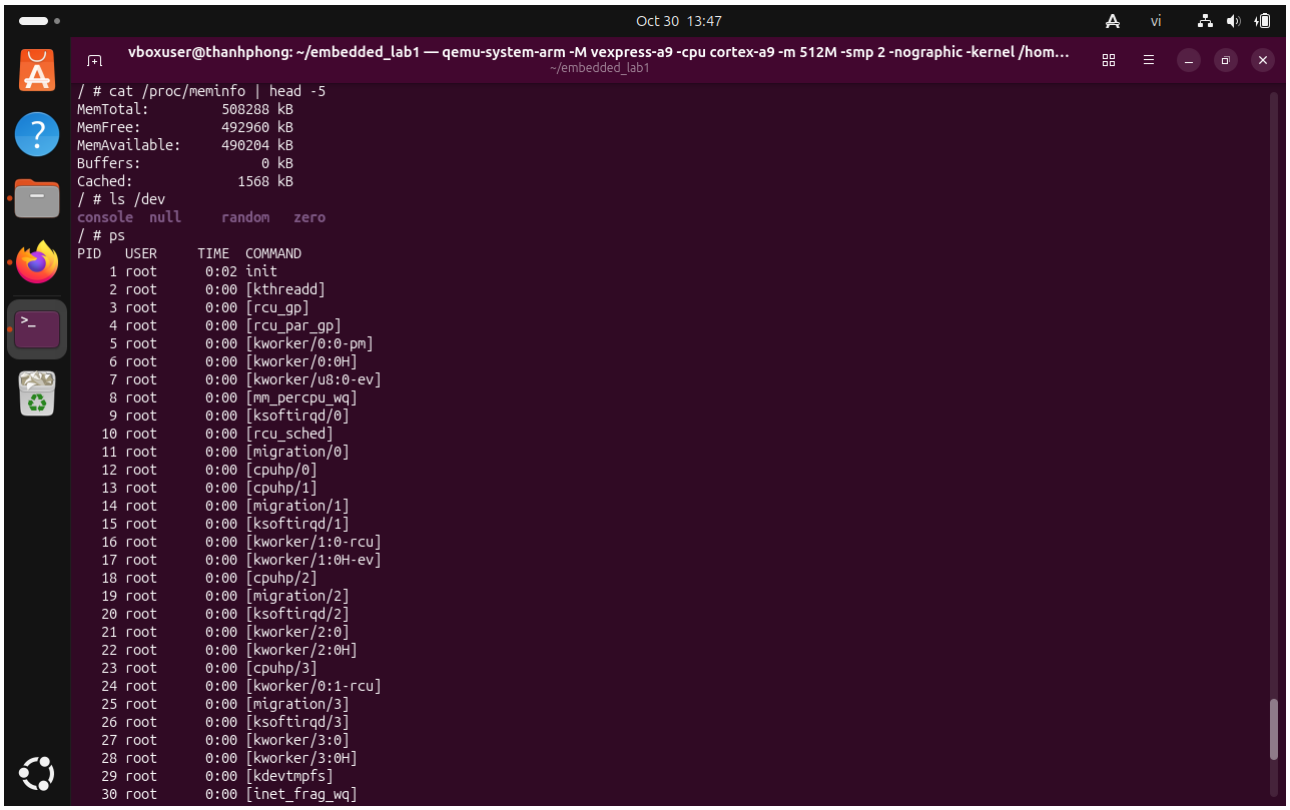
```
/ # uname -a
Linux embedded-lab 5.15.0 #1 SMP Wed Sep 16 01:29:30 UTC 2026 armv7l GNU/Linux
/ # cat /proc/cpuinfo
processor       : 0
model name      : ARMv7 Processor rev 0 (v7l)
BogoMIPS        : 1391.00
Features        : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
CPU implementer : 0x41
CPU architecture: 7
CPU variant     : 0x0
CPU part        : 0xc09
CPU revision    : 0

processor       : 1
model name      : ARMv7 Processor rev 0 (v7l)
BogoMIPS        : 1777.66
Features        : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
CPU implementer : 0x41
CPU architecture: 7
CPU variant     : 0x0
CPU part        : 0xc09
CPU revision    : 0

Hardware        : ARM-Versatile Express
Revision        : 0000
Serial          : 0000000000000000
/ #
```

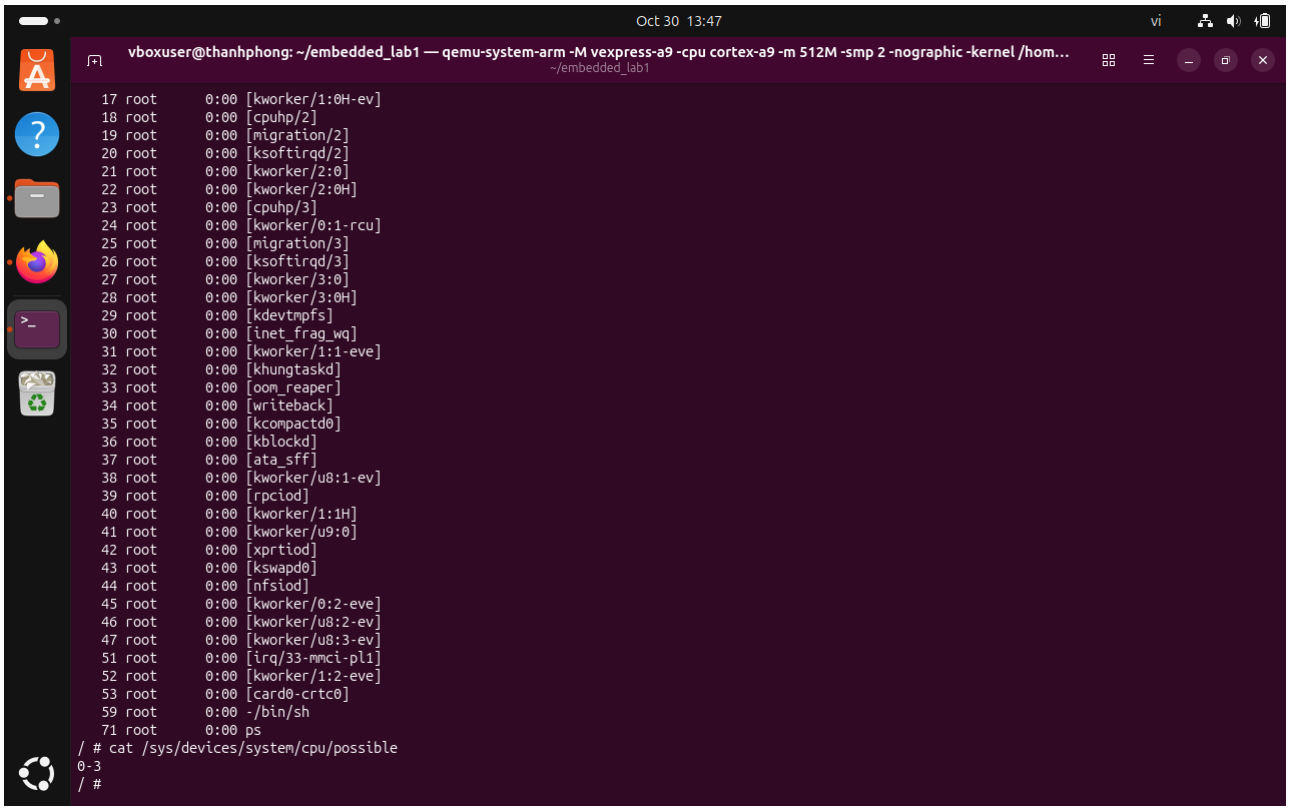
Nguồn: Lab01_PhanC_01_Kernel_CPU.png

C2a. Kiểm tra hệ thống sau boot



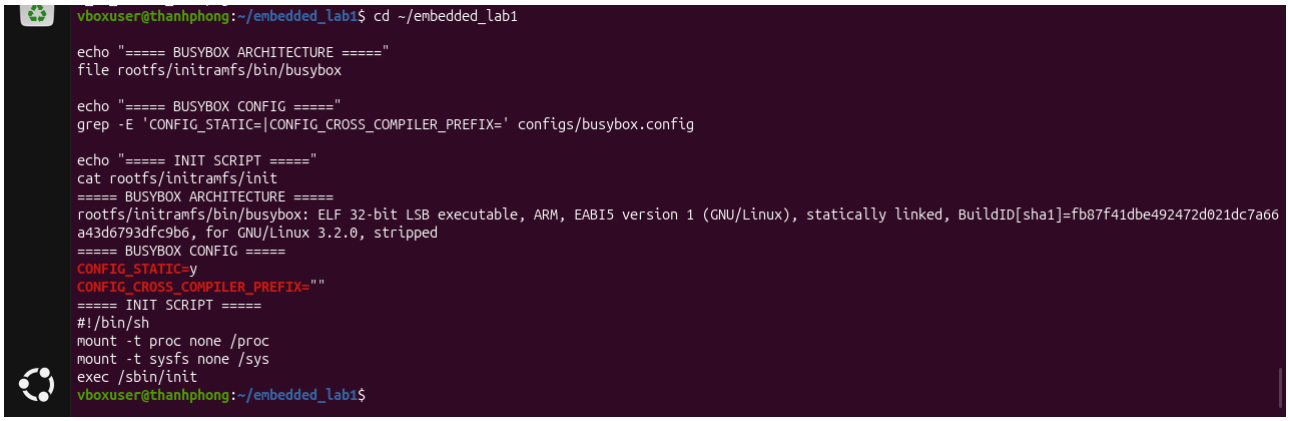
Nguồn: Lab01_PhanC_05_System_Check1.png

C2b. Các tiến trình và số CPU được hỗ trợ



Nguồn: Lab01_PhanC_05_System_Check2.png

C3. BusyBox static và script /init



Nguồn: Lab01_PhanC_06_BusyBox_Config.png